IT550 Project: Latent Profiles for Oblique Retrieval

Oct 20, 2026 · @Jeel Prajapati

Summary

We make search work for **oblique queries**: searches where a document is relevant because of a hidden property, like writing style or a proof technique, not because of its topic. The base paper is OBLIQ-Bench (MIT, May 2026). It shows that every practical retriever scores below about 0.2 nDCG@10, while an LLM that is shown the right documents recognises them easily.

What we build. Before searching, a small open LLM reads every document once and writes a short **latent profile**: the hidden traits it shows. We index these profiles next to the original text, profile each query the same way, and fuse both result lists. A small LLM can optionally re-rank the top results.

Why it is new. The paper tried fixes on the *query* side only, query rewriting and a multi-hop agent, and found they do not solve the problem. Query rewriting even hurt on Writing-Style (0.164 → 0.018). We test the untried other side: making the hidden traits of *documents* searchable. The paper itself names this as an open direction.

One-line pitch: “Retrievers match topics; we teach them to match hidden traits by having an LLM write them down at indexing time.”

Team: Jeel Prajapati (202301238) · Rudra Desai (202301255) · IT550 Information Retrieval, Prof. Sandip Modha

The problem: oblique queries

Normal search finds documents that are **about the same topic** as the query. Oblique queries need documents that **share a hidden property** the text never states, so topic matching misses them.

Kind (paper’s term)	Everyday example	Why normal search fails
Descriptive	“Find tweets that quietly support side X”	The tweet never says “I support X”; it implies it

Kind (paper's term)	Everyday example	Why normal search fails
Analogue	"Find other essays by the author of this one" or "Find problems solved with the same trick"	The topic is different; only the style or method is shared
Tip-of-the-tongue	"That hearing where a senator got angry about a tech CEO dodging a question"	The memory is vague, with no names or dates to match

Why retrievers fail. BM25 matches words, and embedding models compress a text into one vector that mostly captures its topic. Traits like tone, stance, sentence rhythm or proof technique barely move that vector, so two essays by the same author on different topics end up far apart.

The key finding: retrieval–verification asymmetry. When GPT-5.2 is given a candidate list that contains the right documents, it finds them well (nDCG@10 0.33–0.91). A retriever on its own almost never puts them in that list. The bottleneck is the first search step, not the LLM's judgement. That is exactly the step we target.

Benchmark: OBLIQ-Bench

OBLIQ-Bench has 5 tasks, released on Hugging Face under CC-BY-4.0 (dataset). We focus on **Math** and **Writing-Style**. They are small enough to profile every document with a free LLM, and Writing-Style's labels come from real authorship, with no LLM involved.

Task	Queries	Corpus	Query	Relevant document	Labels	Our use
Writing-Style	512	~10.9k snippets	A snippet by one author	Other snippets by the same author, different topic	Real authorship (no LLM)	**Core**
Math Meta-Program	151	~3.66k problems	Abstract description of a proof technique	A problem on another topic using that technique	GPT-5, clustered	**Core**
Twitter-Conflict	281	~72.4k tweets	A stance description	A tweet implying that stance	GPT-5, re-judged	Stretch
Congress Hearings	254	~214k passages	Vague memory of a hearing moment	The one exact passage	One known gold	Optional
WildChat Errors	40	~508k chats	A named AI failure mode	A chat showing it	GPT-5.4-nano + GPT-5	Skip (too large)

Files in each task folder

- `corpus.jsonl` : one document per line (`_id` , `text`)
- `queries.jsonl` : one query per line (`_id` , `text` , task metadata)
- `qrels.tsv` : gold relevance judgments (`query_id` , `corpus_id` , `score`)
- `qrels_pool.tsv` : extra judgments from the authors' pooled evaluation (some tasks)
- `per_query_excluded_ids.json` (Math, Writing): documents to **remove from each query's results** before scoring. For example, Writing-Style hides other snippets from the same source post, so the task can't be solved by near-duplicates. Skipping this mask inflates scores and makes our numbers incomparable.

Results to compare against

The target is to beat the paper's best single-stage retriever: **0.147 on Math** and **0.164 on Writing-Style** (nDCG@10). We will use only free, open models, while the paper used Gemini embeddings and GPT-5.2.

nDCG@10 from the paper (Math and Twitter use pooled judgments; Writing and Congress have a single gold document per query)

System	Math	Writing	Twitter	Congress
BM25	.029	.077	.002	.000
LateOn 0.1B (multi-vector)	.128	.105	.010	.083
Qwen3-Embedding-0.6B	.143	.046	.012	.006
Qwen3-Embedding-4B	.129	.033	.039	.040
Gemini-2-Embedding	.147	.164	.132	.059
GPT-5.2 query rewriter + Gemini	.185	.018	.139	.084
GPT-5.2 multi-hop agent (4 rounds)	.207	.061	.215	.183
Oracle GPT-5.2 reranker (gold injected)	.329	.515	.436	.913

The last row is an upper bound, not a real system: the right documents were added to the candidate list before re-ranking.

What the systems are

- **BM25** uses default settings in the `rank-bm25` Python library.
- **LateOn** is a 149M-parameter late-interaction (ColBERT-style) model, indexed with PyLate.
- **Query rewriter**: GPT-5.2 rewrites the query once, then Gemini retrieves the top 1,000.

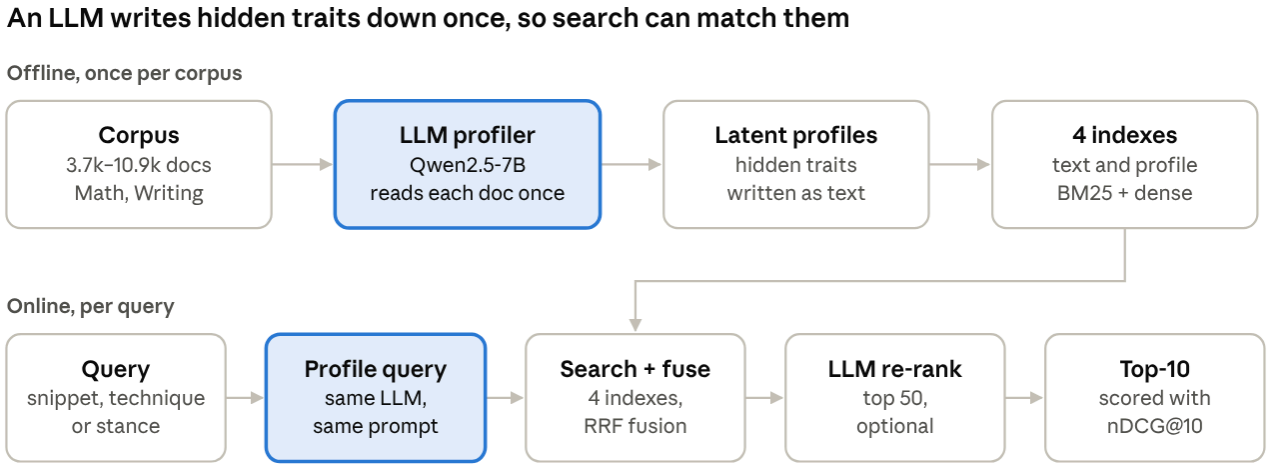
- **Multi-hop agent**: four rounds. Each round GPT-5.2 writes a search, reads the top 25, picks candidates and takes notes.

Rows we will reproduce ourselves: BM25, Qwen3-Embedding-0.6B and, if time allows, LateOn. Matching their numbers within about ±0.01 shows our pipeline and evaluation are correct before we add our method.

For Writing-Style, the table text I read was not clearly labelled beyond the first metric column. Quote only nDCG@10 until you have checked Table 2 in the PDF.

Our approach: latent profiles

We move the LLM's understanding to indexing time. It runs once per document, offline, so each search stays cheap.



latent-profile pipeline · offline and online paths

The highlighted boxes are the new parts. Everything else is a standard hybrid search pipeline, which doubles as our baseline.

Example profile (Math). A problem's raw text talks about triangles; its profile says "proof by contradiction; extremal principle; assumes a minimal counterexample." A query describing "assume the smallest bad case and derive a smaller one" now shares words and meaning with that profile, even though the topics differ.

Example profile (Writing-Style). "Short declarative sentences; frequent em-dashes; first person; rhetorical questions; dry humour; technical vocabulary from software." Two snippets by the same author on different topics get similar profiles, even though their topics share no words.

Variants we compare

1. **P-only:** search the profile indexes alone.

2. **Text + profile fusion:** RRF over all four indexes. This is our main system.
3. **Fusion + small-LLM re-rank:** a cheap open-model version of the paper's oracle reranker, with no gold documents injected.

Implementation

Everything runs on free resources. A laptop handles BM25, fusion and evaluation; a Kaggle GPU (2× T4, about 30 hours a week) handles LLM profiling and dense encoding. Every LLM output is cached to disk, so it is computed only once.

Stack

Part	Tool	Notes
Data loading	`datasets` / `huggingface_hub`	Download `math/` and `writing/` folders from `dianetc/OBLIQ-Bench`
BM25	`rank-bm25` (same as the paper)	Pyserini is optional, for speed
Dense retrieval	`Qwen/Qwen3-Embedding-0.6B` + FAISS (exact inner product)	Same model as the paper's row, so we can reproduce it
Multi-vector (optional)	`lightonai/LateOn` + PyLate	Reproduces the paper's LateOn row
Profiling LLM	`Qwen2.5-7B-Instruct` (AWQ 4-bit) via vLLM	Fallbacks: Qwen2.5-3B, or Llama-3.1-8B on a free API tier
Re-ranker (optional)	Same Qwen2.5-7B, listwise prompt	Or a small cross-encoder reranker
Fusion and metrics	`ranx`	RRF fusion, nDCG@10, Recall@k, paired significance tests

Step by step

1. **Load and mask.** Read `corpus.jsonl`, `queries.jsonl`, `qrels.tsv` and `qrels_pool.tsv`. Load `per_query_excluded_ids.json` and drop those IDs from every result list before scoring. Write one helper, `mask(run)`, and run every system through it.
2. **Reproduce the baselines.** Run BM25 and Qwen3-Embedding-0.6B on the raw text. Target: within about ±0.01 of the paper (Math .029 / .143 pooled; Writing .077 / .046). If you are off, fix the evaluation before going further.

3. **Design the profile prompts.** Write one prompt per task and test it by hand on 20 documents. Keep the output short (60–120 words) and in a fixed format, so profiles are comparable.
4. **Profile every document.** Batch-run the LLM with vLLM and save `profiles.jsonl` (`_id`, `profile`). Time it on 200 documents first, then run the rest. Our rough estimate is 1–2 GPU hours per task; measure it before relying on it.
5. **Profile every query** with the same prompt. A Writing query is itself a snippet; a Math query is a technique description, so its prompt asks "what techniques would a solution need?".
6. **Build four indexes.** BM25 and dense on the text; BM25 and dense on the profiles.
7. **Retrieve and fuse.** Take the top 1,000 from each index and combine them with RRF (k = 60). Also try weighted fusion, with weights tuned on a small held-out split of queries.
8. **Optional re-rank.** A listwise LLM prompt re-orders the top 50 of the fused list, in windows of 10–20.
9. **Evaluate.** Apply the mask, then compute nDCG@10 (main metric), nDCG@50, Recall@10/50/100, with both gold and pooled qrels for Math.

Example profile prompts

Writing-Style document prompt

```
Describe the WRITING STYLE of the passage below, not its topic.
Cover: sentence length and rhythm, punctuation habits, person (I/we/you),
tone, humour, typical phrases, vocabulary register, paragraph structure.
Do NOT mention the subject matter, names, or facts.
Answer in 60-100 words as a comma-separated list of style traits.

Passage:
{text}
```

Math document prompt

```
Read the math problem and its solution. Name the REASONING TECHNIQUES used,
not the topic (e.g. "pigeonhole principle", "invariant", "extremal
principle",
"induction on n", "proof by contradiction", "bijection", "double counting").
Then describe the key move in one abstract sentence without any topic words.
Answer in under 100 words.

Problem and solution:
{text}
```

The “do not mention the topic” instruction is the core of the method. It forces the profile to capture the hidden traits instead of repeating the topic, which the retriever already sees.

Code sketch

```
from rank_bm25 import BM25Okapi
from ranx import Qrels, Run, fuse, evaluate

# runs: dict[qid] -> dict[doc_id] -> score, top 1000, already masked
runs = {
    "bm25_text":     Run(bm25_search(text_index, queries)),
    "dense_text":    Run(dense_search(text_emb, query_emb)),
    "bm25_profile":  Run(bm25_search(profile_index, query_profiles)),
    "dense_profile": Run(dense_search(profile_emb, query_profile_emb)),
}
fused = fuse(list(runs.values()), method="rrf")
print(evaluate(Qrels.from_file("qrels.tsv", kind="trec"), fused, ["ndcg@10",
"recall@100"]))
```

The qrels files are tab-separated with a header (`query_id` , `corpus_id` , `score`). Convert them to ranx's format once, rather than assuming the TREC layout shown above.

Experiments and evaluation

The main claim to test: **adding LLM document profiles to a free, open retriever raises nDCG@10 on Math and Writing-Style above the paper's best single-stage retriever (0.147 and 0.164).**

Research questions

1. **RQ1:** Can we reproduce the paper's BM25 and Qwen3-Embedding-0.6B numbers?
2. **RQ2:** Do profiles alone retrieve better than raw text?
3. **RQ3:** Does fusing text and profiles beat either one alone?
4. **RQ4:** Does document-side profiling beat query-side rewriting, especially on Writing-Style where the paper's rewriter dropped to 0.018?
5. **RQ5 (optional):** How much of the gap to the oracle (0.329 / 0.515) does a small open-model re-ranker close?

Experiment grid

ID	System	Answers
E1	BM25 text, Qwen3-0.6B text (reproduction)	RQ1
E2	Our own query rewriter (same open LLM) + Qwen3 text	RQ4, a fair same-LLM comparison
E3	BM25 profile, Qwen3 profile	RQ2
E4	RRF over all four indexes (main system)	RQ3
E5	E4 + LLM re-rank of the top 50	RQ5
A1	Ablation: profile prompt *with* topic words allowed	Is “no topic words” what makes it work?
A2	Ablation: smaller profiler (3B vs 7B)	How much LLM size matters
A3	Ablation: profile length (40 vs 100 words)	Cost vs quality

Rules for fair numbers

- Apply the per-query exclusion mask to every run.
- Report gold *and* pooled nDCG@10 for Math, as the paper does. Note that pooled qrels were built from the paper's systems, so a new system that finds unjudged relevant documents can be under-scored. Say this in the report.
- Tune fusion weights and prompts on a fixed 20% dev split of queries; report on the remaining 80% and on all queries.
- Run a paired t-test (or randomisation test) per query against the best baseline; claim a win only at $p < 0.05$.
- Report cost: GPU minutes to profile the corpus and seconds per query.

Analysis to include

- 10 queries where profiles helped most and 10 where they hurt, each with its profile text.
- For Writing-Style, a check that profiles of the same author are closer to each other than to other authors. A small cosine-similarity histogram is enough.

Team split and work plan

The plan fits four weeks, from 20 October to mid-November. The final deadline still needs confirming. Person A owns retrieval and evaluation; Person B owns everything the LLM does. They share one file format: run files of `qid → {doc_id: score}`.

Week	Person A: retrieval and evaluation	Person B: LLM side	Done when
1 (20–26 Oct)	Load data, exclusion mask, BM25 and Qwen3-0.6B baselines, ranx evaluation	Set up vLLM on Kaggle; draft and hand-test profile prompts on 20 docs per task	E1 matches the paper within ±0.01
2 (27 Oct–2 Nov)	Build profile indexes; RRF and weighted fusion; dev/test query split	Profile all Math and Writing documents and queries; same-LLM query rewriter (E2)	E2–E4 numbers exist
3 (3–9 Nov)	Significance tests, cost measurement, ablation runs	LLM re-ranker (E5); ablations A1–A3; error analysis	Full results table and analysis
4 (10–16 Nov)	Results section, figures, reproducible repo	Method and analysis sections, slides	Report and slides submitted

If time allows, add Twitter-Conflict (72k tweets) in week 3. It needs about 7× more profiling than Writing-Style.

Risks and fallbacks

Risk	Early sign	Fallback
Profiles don't beat the baselines	E3/E4 below E1 on the dev split in week 2	Report it as a finding with error analysis. A careful negative result on a 2026 benchmark is still a valid project
Profiles leak the topic anyway	Profiles mention subjects despite the prompt	Tighten the prompt, add a filter that strips topic nouns, and use ablation A1 to show the effect
Can't reproduce the paper's baselines	E1 off by more than 0.02	Check the exclusion mask, the qrels type (gold vs pooled) and the BM25 tokenisation; ask the authors if still stuck

Risk	Early sign	Fallback
Math documents may not include solutions	The corpus text has problems only	Profile from the problem text; mention that the paper's oracle got a large boost from solutions (.329 → .473)
GPU time runs out	Profiling 200 docs takes over 5 minutes	Switch to Qwen2.5-3B, shorten profiles, or use a free API tier
Pooled qrels under-score new systems	Many top results from our system are unjudged	Report the gold-only nDCG too, and hand-judge a small sample of unjudged hits
It's a preprint	—	Cite it as an arXiv preprint (May 2026). Writing-Style labels come from real authorship, so lead with that task

Form answers

Field	Answer
Based on	Research Paper
Paper title	OBLIQ-Bench: Exposing Overlooked Bottlenecks in Modern Retrievers with Latent and Implicit Queries
Conference	Other: arXiv preprint, 2026 (MIT)
Link	https://arxiv.org/abs/2605.06235
Project title	Latent Profiles: LLM-Written Document Traits for Retrieving Oblique Queries

Scope (2–3 lines)

> Modern retrievers fail on “oblique” queries, where relevance depends on a hidden property such as writing style or proof technique rather than topic. On OBLIQ-Bench (2026), we reproduce BM25 and dense baselines, then propose LLM-written latent profiles of each document at indexing time, fused with standard retrieval and an optional small-LLM re-ranker. We measure the nDCG@10 gain on the Math and Writing-Style tasks against the paper's published results.

References

Read the four marked ★ first. The rest support specific sections of the report.

Base paper and benchmark

- ★ Tchuindjo, Shah & Khattab. OBLIQ-Bench: Exposing Overlooked Bottlenecks in Modern Retrievers with Latent and Implicit Queries. arXiv preprint, May 2026.
- ★ OBLIQ-Bench dataset on Hugging Face (CC-BY-4.0).

Closely related: why retrievers miss hidden properties

- Weller, Boratko, Naim & Lee. On the Theoretical Limitations of Embedding-Based Retrieval. ICLR 2026. Single-vector embeddings provably can't represent every set of relevant documents · LIMIT data
- Su et al. BRIGHT: A Realistic and Challenging Benchmark for Reasoning-Intensive Retrieval. ICLR 2025. OBLIQ-Bench compares itself to BRIGHT · code

Our method's lineage: expanding documents before indexing

- ★ Nogueira et al. Document Expansion by Query Prediction (Doc2Query). 2019. The original idea of adding model-generated text to documents before indexing.
- Gospodinov, MacAvaney & Macdonald. Doc2Query--: When Less is More. ECIR 2023. Filtering bad expansions matters, which is relevant to keeping profiles free of topic words.

Query-side LLM methods (what we compare against)

- Gao et al. Precise Zero-Shot Dense Retrieval without Relevance Labels (HyDE). ACL 2023.
- Wang, Yang & Wei. Query2doc: Query Expansion with Large Language Models. EMNLP 2023.
- Sun et al. Is ChatGPT Good at Search? LLMs as Re-Ranking Agents (RankGPT). EMNLP 2023. The listwise re-ranking prompt for E5 · code

Writing style and authorship

- Rivera-Soto et al. Learning Universal Authorship Representations (LUAR). EMNLP 2021. A trained style-embedding model; a useful extra baseline for Writing-Style · code

Models

- ★ Qwen3-Embedding-0.6B · technical report · GitHub
- LateOn: the 149M multi-vector model in the paper's table
- Qwen2.5-7B-Instruct: the profiling LLM (use an AWQ version on a T4)

Libraries and repos

- rank_bm25: the BM25 the paper used
- ranx: metrics, RRF fusion, significance tests
- PyLate: indexing and search for LateOn
- vLLM: fast batched LLM inference for profiling
- sentence-transformers and FAISS: dense encoding and search

Blogs and explainers

- Qwen3 Embedding announcement
- DenseOn with the LateOn: LightOn's post on the LateOn model
- Multi-vector (late interaction) models with Sentence Transformers: Hugging Face blog
- Document expansion: overcoming vocabulary mismatch: a gentle intro to Doc2Query-style methods

What I verified: I opened the OBLIQ-Bench paper and its dataset card for every number and file name in this doc. The other links are checked to exist; skim them before citing specific claims.